

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler

# Load dataset
data = pd.read_csv("/content/weatherHistory.csv")

# Display first five rows
print(data.head())

# Check missing values
print(data.isnull().sum())

# Remove missing values (if any)
data = data.dropna()

# Convert categorical columns into numbers
categorical_cols = ['Summary', 'Precip Type', 'Daily Summary']
for col in categorical_cols:
    encoder = LabelEncoder()
    data[col] = encoder.fit_transform(data[col])

# Drop 'Formatted Date' as it's not directly usable as a numerical feature with
data = data.drop("Formatted Date", axis=1)

# Features (all columns except the target)
# Let's choose 'Temperature (C)' as the target for a regression task
X = data.drop("Temperature (C)", axis=1)

# Target
y = data["Temperature (C)"]

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Standardization
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

```

	Formatted Date	Summary	Precip Type	Temperature (C)
0	2006-04-01 00:00:00.000 +0200	Partly Cloudy	rain	9.472222
1	2006-04-01 01:00:00.000 +0200	Partly Cloudy	rain	9.355556
2	2006-04-01 02:00:00.000 +0200	Mostly Cloudy	rain	9.377778
3	2006-04-01 03:00:00.000 +0200	Partly Cloudy	rain	8.288889
4	2006-04-01 04:00:00.000 +0200	Mostly Cloudy	rain	8.755556

	Apparent Temperature (C)	Humidity	Wind Speed (km/h)	\
0	7.388889	0.89	14.1197	
1	7.227778	0.86	14.2646	
2	9.377778	0.89	3.9284	

3	5.944444	0.83	14.1036
4	6.977778	0.83	11.0446

	Wind Bearing (degrees)	Visibility (km)	Loud Cover	Pressure (millibars)
0	251.0	15.8263	0.0	1015.13
1	259.0	15.8263	0.0	1015.63
2	204.0	14.9569	0.0	1015.94
3	269.0	15.8263	0.0	1016.41
4	259.0	15.8263	0.0	1016.51

Daily Summary

0 Partly cloudy throughout the day.

1 Partly cloudy throughout the day.

2 Partly cloudy throughout the day.

3 Partly cloudy throughout the day.

4 Partly cloudy throughout the day.

Formatted Date 0

Summary 0

Precip Type 517

Temperature (C) 0

Apparent Temperature (C) 0

Humidity 0

Wind Speed (km/h) 0

Wind Bearing (degrees) 0

Visibility (km) 0

Loud Cover 0

Pressure (millibars) 0

Daily Summary 0

dtype: int64

```
import torch
import torch.nn as nn

# Convert data into tensors

X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32)

y_train_tensor = torch.tensor(y_train.values, dtype=torch.float32).view(-1,1)
y_test_tensor = torch.tensor(y_test.values, dtype=torch.float32).view(-1,1)

# Build model

model = nn.Sequential(
    nn.Linear(X_train.shape[1], 16),
    nn.ReLU(),
    nn.Linear(16, 8),
    nn.ReLU(),
    nn.Linear(8, 1)
)
```

```
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

epochs = 100
```

```
losses = []

for epoch in range(epochs):

    outputs = model(X_train_tensor)

    loss = criterion(outputs, y_train_tensor)

    optimizer.zero_grad()

    loss.backward()

    optimizer.step()

    losses.append(loss.item())

    if (epoch+1)%10==0:
        print(f"Epoch {epoch+1}, Loss={loss.item():.4f}")
```

```
Epoch 10, Loss=218.2621
Epoch 20, Loss=169.7317
Epoch 30, Loss=76.4891
Epoch 40, Loss=26.6226
Epoch 50, Loss=17.3038
Epoch 60, Loss=10.1879
Epoch 70, Loss=5.8770
Epoch 80, Loss=3.8902
Epoch 90, Loss=2.7815
Epoch 100, Loss=2.3139
```

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

model.eval()

with torch.no_grad():
    predictions = model(X_test_tensor)

predictions = predictions.numpy()

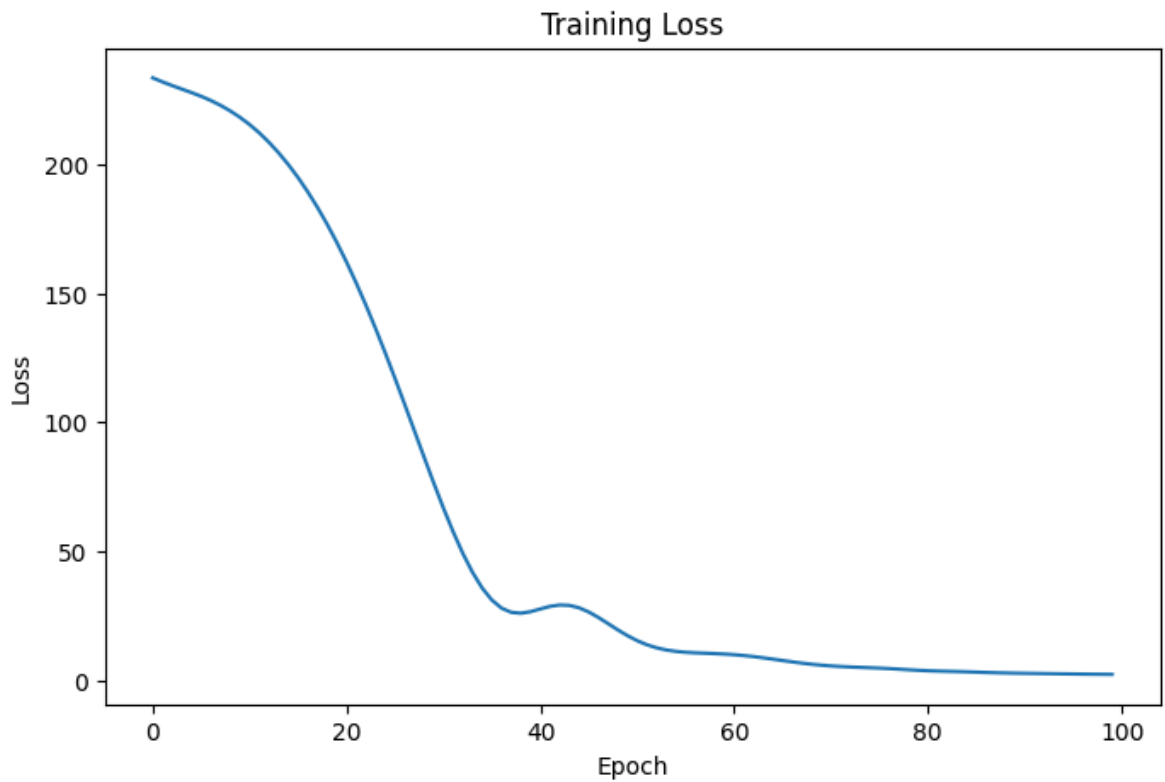
mse = mean_squared_error(y_test, predictions)
rmse = mse**0.5
mae = mean_absolute_error(y_test, predictions)
r2 = r2_score(y_test, predictions)

print("MSE :", mse)
print("RMSE:", rmse)
print("MAE :", mae)
print("R2 Score:", r2)
```

```
MSE : 2.2701619021321493
RMSE: 1.5067056454835992
MAE : 1.0668174029220479
R2 Score: 0.9751030613246569
```

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(8,5))  
plt.plot(losses)  
plt.xlabel("Epoch")  
plt.ylabel("Loss")  
plt.title("Training Loss")  
plt.show()
```



```
plt.figure(figsize=(6,6))  
plt.scatter(y_test, predictions)  
plt.xlabel("Actual")  
plt.ylabel("Predicted")  
plt.title("Actual vs Predicted")  
plt.show()
```

